

Sorting Lower Bound (Comparison Based Sorting)

25/8/26

We'll now focus on counting only the no of comparisons.

Th.
 $\Leftrightarrow$

Comparison Based Sorting can't be solved in $\Omega(n \log n)$ time.

$\nexists$ any comparison based sorting algo that runs in $\Omega(n \log n)$ time.

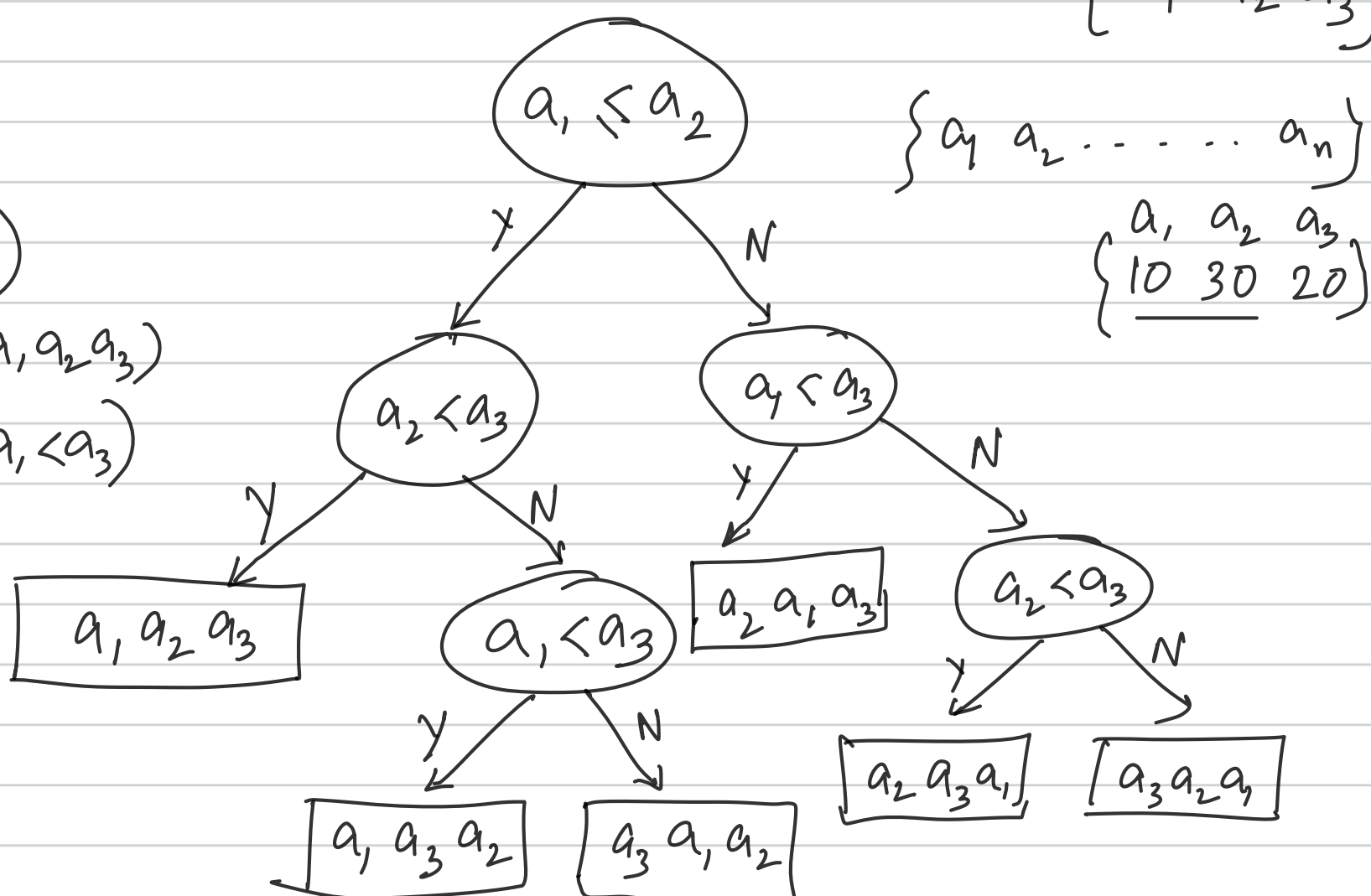
$$\begin{aligned} T(n) &= \Omega(n \log n) \\ T(n) &\geq c \cdot n \log n \end{aligned}$$

Any comparison based algo can be viewed as a tree of all possible comparisons & their outcomes and the resulting answer. This tree is called "Decision Tree".

If ($a_1 < a_2$)

If ($a_2 < a_3$)
then (a_1, a_2, a_3)
Else If ($a_1 < a_3$)
:
:

Else



Decision Tree

Algorithm.

Internal node



Comparison

leaf



Answer

Path from root to a leaf



An execution of algo.

length of that path

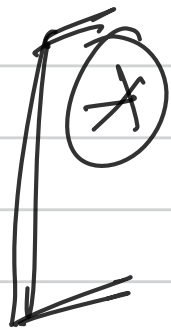


Running time for that execution

height



Running time of the algo
(Worst Case)



Any decision tree that represents a ^{comparison based} sorting algo has height at least $n \log n$

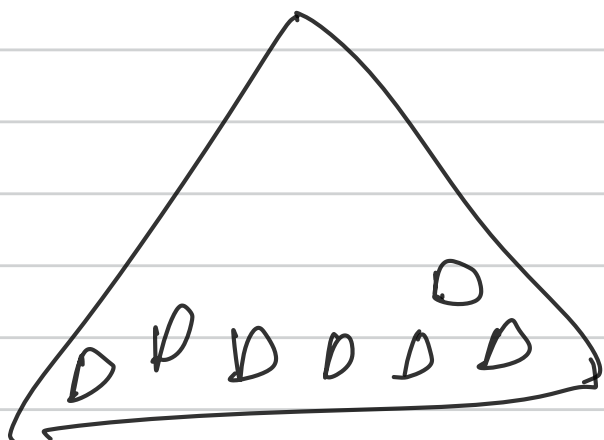
Obs

Every correct algo must have all the possible answer in the decision tree.

For sorting, how many possible soln $= n!$

Decision tree of height h .

How many leaves are there ?? $= 2^h$.



Each $n!$ permutations of n elements must appear as one of the leaves.

$$\Rightarrow 2^h \geq n!$$

$$\Rightarrow h \geq \log_2 n!$$

$$\Rightarrow h = \Omega(n \log n)$$

$$\begin{aligned}
 n! &= n \cdot (n-1) \cdot (n-2) \cdots \left(\frac{n}{2}\right) \cdots 1 \\
 &> n \cdot (n-1) \cdot (n-2) \cdots \left(\frac{n}{2}\right) \\
 &> \underbrace{\frac{n}{2} \cdot \frac{n}{2} \cdot \frac{n}{2} \cdots \frac{n}{2}}_{\frac{n}{2} \text{ many}} = \left(\frac{n}{2}\right)^{\frac{n}{2}}
 \end{aligned}$$

$$\begin{aligned}
 \Rightarrow \log n! &> \frac{n}{2} \log \frac{n}{2} \\
 &= \Omega(n \log n)
 \end{aligned}$$

Sorting in linear time. ($\underline{O(n)}$) (Integer Sorting)
 (Non comparison based algo)

Counting Sort:

Assumption: n integers $\in \{1, 2, \dots, k\}$

Eg. marks of 100000000 students.
 (out of 100)

$$\{a_1, a_2, \dots, a_{100000000}\} \quad a_i \in \{0, \dots, 100\}$$

Counting Sort

for $i=1$ to n

Insert a_i in correct queue.

$$A[1] \rightarrow a_1, a_{10}, a_{15}, \dots \quad (\boxed{n_1})$$

$$A[2] \rightarrow \quad (\boxed{n_2})$$

$$A[52] \rightarrow a_i$$

$$A[k] \rightarrow \quad (\boxed{n_k})$$

for each $j=1$ to k

print the list $A[j]$

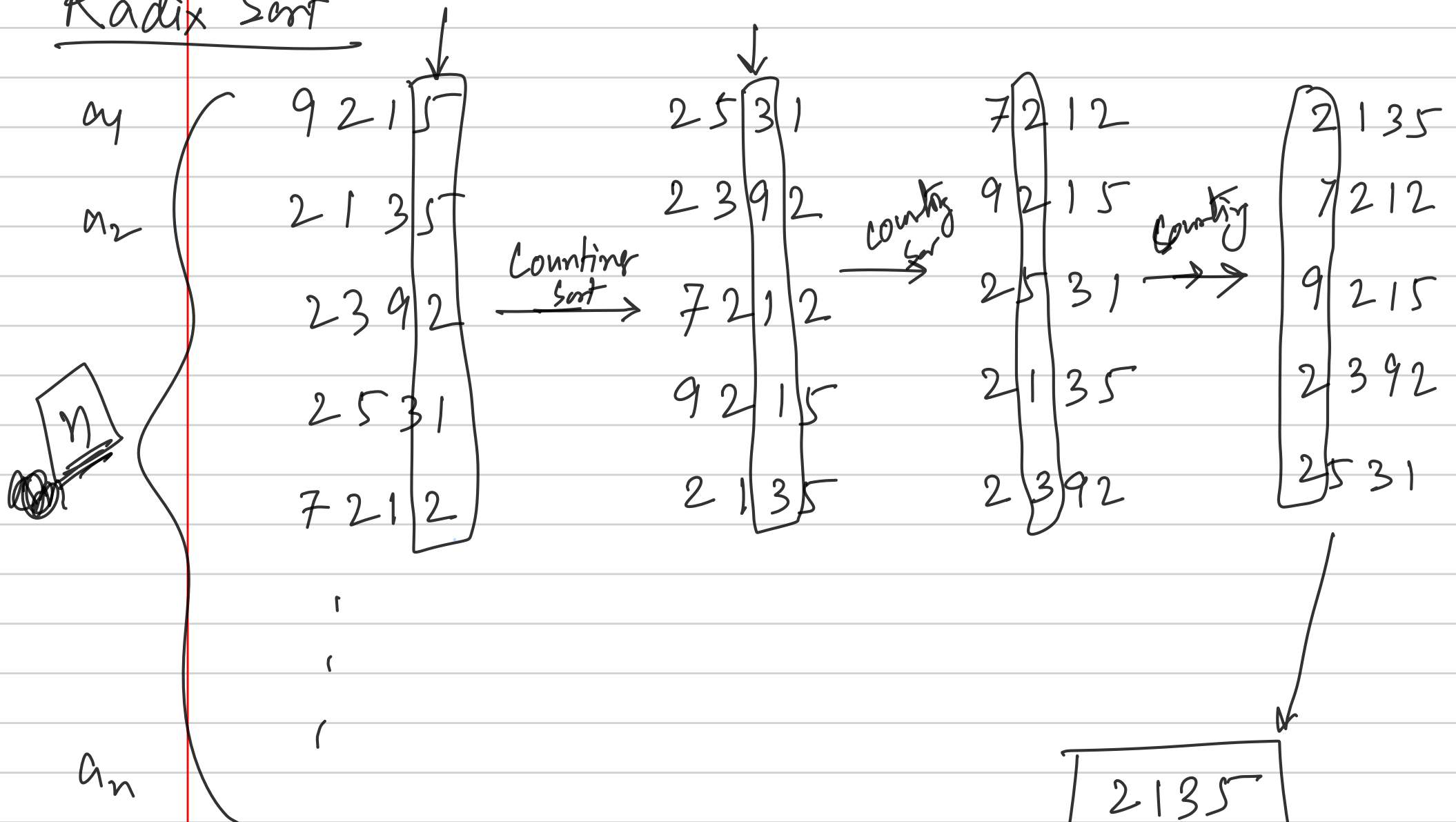
$O(n+k)$



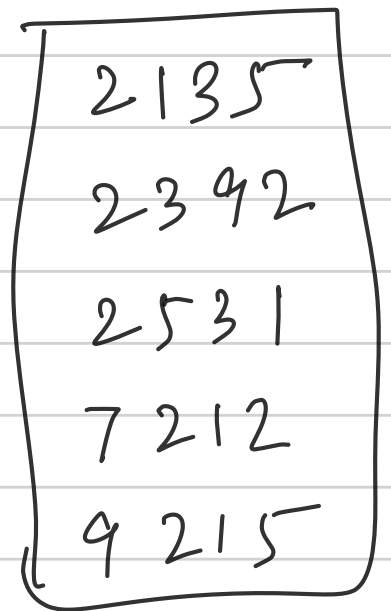
$$\begin{aligned}
 &(\boxed{n_1} + 1) + (\boxed{n_2} + 1) + \dots + (\boxed{n_k} + 1) \\
 &= \sum n_i + k = n + k
 \end{aligned}$$

When $k = O(n)$, then Running time of Counting sort is linear $O(n)$.

Radix Sort



Ex Do the similar process but starting from 1000th place till unit place.



Stable Property: Relative ordering of the input is maintained in the output

Input: An array of n d -digit nos and each digit can take upto k diff values.

~~Q1~~

Radix Sort

for $i = 1$ to d

$1 \rightarrow$ least significant digit
 $d \rightarrow$ most " "

Apply counting sort on i -th digit of the nos.

Running time . $O(d * O(n+k)) = \underline{O(d(n+k))}$

For $d = O(1)$ and $k = O(n)$, Running time = $O(n)$
linear.